

HELPGRID

Detailed Technical Requirements Document

Technical blueprint for a local problem-to-solution coordination platform

Version: 1.0 Frontend: React + Tailwind Backend: Node.js + Express Database: PostgreSQL

This TRD translates the HelpGrid PRD into an implementable technical design. It defines system architecture, service boundaries, domain models, API contracts, AI processing, matching, realtime events, security, deployment, testing and engineering conventions.

Layer	Primary technology
Web application	React, React Router, Tailwind CSS
API server	Node.js + Express
ORM	Prisma
Database	PostgreSQL
Realtime	Socket.IO
Maps	Mapbox or Google Maps
Media	Cloudinary or S3-compatible object storage
AI	LLM API with structured-output validation + deterministic fallback
Deployment	Frontend CDN/hosting + backend service + managed PostgreSQL

Technical objective

Build a modular system where the Request is the source of truth, AI is an assistive processing layer, matching is explainable, realtime is an enhancement over persisted state, and all critical transitions are enforced by the backend.

1. Technical Overview

HelpGrid is a web application with a client-server architecture. The browser handles presentation, navigation, local form state and realtime UI. The backend owns authentication, authorization, request lifecycle rules, matching, persistence, AI orchestration and notifications. PostgreSQL is the source of truth for durable state.

1.1 High-level architecture

```
Browser → React UI → REST API / Socket.IO → Express services → Prisma → PostgreSQL
                                     ↳ AI service
                                     ↳ Maps/geocoding
                                     ↳ Object storage
                                     ↳ Notification provider
```

1.2 Core architectural principles

- Keep domain logic on the server; the client should not implement authoritative task transitions.
- Use PostgreSQL transactions for critical state changes.
- Use Socket.IO for event delivery, but persist the state before emitting the event.
- Keep AI outputs structured and schema-validated.
- Make matching deterministic and explainable in the first version.
- Keep external providers behind small service adapters so they can be replaced.
- Centralize shared request/status types to prevent frontend/backend vocabulary drift.

2. System Components

Component	Responsibility	Key Interfaces
Web Client	Screens, forms, navigation, map, realtime UI	REST, Socket.IO
API Server	Auth, requests, tasks, matches, messages	HTTP JSON
Auth Module	Signup, login, sessions/tokens, role checks	Auth middleware
Request Module	CRUD, validation, lifecycle	Request service
AI Module	Understanding, duplicate check, summaries	AI provider adapter
Matching Module	Eligibility, scoring, ranking	Match service
Task Module	State machine, ETA, arrival, completion	Task service
Messaging Module	Conversation persistence and delivery	Message service + Socket.IO
Media Module	Proof uploads and metadata	Object storage adapter
Notification Module	In-app events and external delivery	Notification adapter
Map Module	Geocoding, distance, map configuration	Map provider adapter
Database	Durable relational source of truth	Prisma/PostgreSQL

2.1 Suggested backend modules

src/modules/auth, users, requests, ai, matching, tasks, messages, proof, verification, notifications, maps, live, health. Each module should expose controller → service → repository/data-access boundaries where complexity warrants them.

3. Frontend Technical Requirements

3.1 Route structure

```
/
/explore
/login
/signup
```

```

/onboarding/role
/onboarding/skills
/app
/app/create
/app/create/details
/app/create/location
/app/create/review
/app/create/success
/app/explore/map
/app/request/:id
/app/matches/:id
/app/requests/:id
/app/requests/:id/verify
/app/messages/:id
/helper
/helper/tasks/:id
/helper/tasks/:id/proof
/live

```

3.2 Frontend layers

Layer	Rule
Pages	Route-specific composition only; avoid embedding business logic everywhere.
Components	Reusable visual/interaction units such as RequestCard, Timeline and MatchScore.
Hooks	Encapsulate request loading, task state, messages, availability and map state.
API client	Single configured HTTP client with auth/error handling.
State	Keep server data separate from transient UI state where practical.
Forms	Schema validation before API submission.
Realtime	Socket connection managed centrally; pages subscribe to domain events.
Design tokens	Colors, spacing, radius and typography defined once.

3.3 Suggested project structure

```

src/
app/ routes/ providers/
components/ ui/ maps/ requests/ tasks/ messaging/
features/ auth/ requests/ matching/ helper/ verification/
hooks/ services/ api/ socket/
types/ constants/ utils/
styles/

```

4. Backend Technical Requirements

4.1 Request pipeline

HTTP request → middleware → authentication → authorization → validation → controller → service → repository/Prisma → transaction → response → optional realtime event

4.2 Middleware

- request ID / correlation ID
- authentication
- role/permission check
- schema validation
- rate limiting
- error normalization
- request logging
- CORS and security headers

4.3 Error format

```
{"success":false,"error":{"code":"REQUEST_NOT_FOUND","message":"Request not found","requestId":"..."},"meta":{}}
```

Success responses should use a consistent envelope where useful; list endpoints should include pagination metadata.

5. Database Technical Requirements

PostgreSQL is the primary relational database. Prisma is used for schema management, migrations and type-safe data access.

5.1 Core schema

Table	Important columns	Notes
users	id, email, passwordHash, name, phone, role, city, trustScore, createdAt	Unique email; role enum.
skills	id, name, category	Normalized skill catalog.
user_skills	userId, skillId, level, verified	Composite unique key.
requests	id, creatorId, type, status, title, description, category, urgency, lat, lng, locationText, deadline, affectedCount	Main domain object.
request_ai	id, requestId, modelVersion, summary, categorySuggestion, urgencySuggestion, duplicateScore	AI audit record.
matches	id, requestId, helperId, score, skillFit, distance, availability, experience, trust, explanation	Unique request/helper pair.
tasks	id, requestId, helperId, status, acceptedAt, startedAt, arrivedAt, completedAt	At most one active task per request in MVP.
messages	id, taskId, senderId, body, createdAt, readAt	Task-scoped conversation.
proofs	id, taskId, type, mediaUrl, note, uploadedAt	Object storage URL/key.
verifications	id, requestId, requesterId, outcome, comment, createdAt	Append-only verification history.
availability	userId, isAvailable, radiusKm, updatedAt	Current helper availability.
notifications	id, userId, type, title, body, readAt, createdAt	In-app notification inbox.

5.2 Indexing

- users(email) UNIQUE
- requests(status, createdAt)
- requests(type, status)
- requests(category, status)
- requests(latitude, longitude) or spatial index if PostGIS is adopted
- matches(requestId, score DESC)
- matches(helperId, createdAt DESC)
- messages(taskId, createdAt)
- notifications(userId, readAt, createdAt)

5.3 Optional spatial upgrade

For a larger deployment, PostgreSQL + PostGIS should replace application-only distance filtering. The MVP can calculate Haversine distance in the service layer if scale is small.

6. Domain Enums & State Machines

6.1 Enums

Role: USER | HELPER | ORGANIZATION | ADMIN
RequestType: NEED | GIVE | SERVICE | REPORT
Urgency: LOW | MEDIUM | HIGH | CRITICAL
TaskStatus: ACCEPTED | IN_PROGRESS | ARRIVED | PROOF_SUBMITTED | AWAITING_VERIFICATION | RESOLVED | REWORK_REQUESTED | CANCELLED

6.2 Request transitions

DRAFT → PUBLISHED → MATCHING → MATCHED → ACCEPTED → IN_PROGRESS → PROOF_SUBMITTED → AWAITING_VERIFICATION → RESOLVED

6.3 Invalid transitions

- A requester cannot directly mark a task as PROOF_SUBMITTED.
- A helper cannot mark a task RESOLVED without required proof.
- A user cannot accept a task unless they are eligible and the task is still available.
- A resolved request should be immutable except through an audited dispute/rework path.

7. REST API Specification

Method	Endpoint	Purpose	Auth
POST	/api/auth/signup	Create account	Public
POST	/api/auth/login	Authenticate	Public
POST	/api/auth/logout	End session	User
GET	/api/me	Current user	User
PATCH	/api/me	Update profile	User
GET	/api/requests	Explore requests	Optional
POST	/api/requests	Create request	User
GET	/api/requests/:id	Request detail	User
PATCH	/api/requests/:id	Edit request	Owner
POST	/api/requests/:id/publish	Publish request	Owner
GET	/api/requests/:id/matches	Get matches	Owner/eligible helper
POST	/api/matches/:id/accept	Accept match/task	Helper
GET	/api/tasks/:id	Task detail	Participants
PATCH	/api/tasks/:id/status	Change task status	Participant
GET	/api/tasks/:id/messages	Get messages	Participants
POST	/api/tasks/:id/messages	Send message	Participants
POST	/api/tasks/:id/proof	Create proof metadata	Helper
POST	/api/requests/:id/verify	Verify outcome	Requester
PATCH	/api/helpers/me/availability	Toggle availability	Helper
GET	/api/live	Live aggregated data	Authenticated/Public policy

7.1 Request creation payload

```
{
  "type": "REPORT",
  "description": "Lab 3 PCs won't boot",
  "latitude": 22.7196,
  "longitude": 75.8577,
  "locationText": "College Lab 3"
}
```

7.2 Request response

```
{
  "id": "req_123",
  "type": "REPORT",
  "status": "PUBLISHED",
  "title": "Lab 3 PCs won't boot",
  "urgency": "HIGH",
  "category": "COMPUTER_HARDWARE",
  "affectedCount": 14,
  "location": {
    "lat": 22.7196,
    "lng": 75.8577,
    "label": "College Lab 3"
  }
}
```

8. AI Technical Design

8.1 AI pipeline

User text → sanitize/normalize → prompt/template → LLM → JSON schema validation → deterministic normalization → duplicate check → store AI result → return editable suggestions

8.2 Structured output schema

```
{"category": "COMPUTER_HARDWARE", "urgency": "HIGH", "affectedCount": 14, "keywords": ["PC", "boot", "lab"], "summary": "Multiple lab computers are not booting.", "confidence": 0.91}
```

The backend must validate enums, numeric ranges and string lengths before persisting AI output.

8.3 Prompt responsibilities

- Extract only information supported by the user's text.
- Return null when information is missing instead of guessing.
- Map free-form descriptions to the application's controlled taxonomy.
- Return concise summaries suitable for UI.
- Never decide verification or completion on behalf of the requester.

8.4 Fallback behavior

If AI is unavailable, request creation must still work. Use keyword/category rules, default urgency MEDIUM and explicit user confirmation. The AI tag should not appear when the value came from deterministic fallback.

9. Duplicate Detection

Duplicate detection prevents five users from reporting the same local issue independently.

9.1 Algorithm

- 1) Find active requests within radius R.
- 2) Compare category/type.
- 3) Compare normalized text embeddings or keyword similarity.
- 4) Combine semantic similarity + spatial proximity + category match.
- 5) Show warning when threshold is exceeded.

9.2 Suggested initial rule

DuplicateScore = 50% semantic similarity + 30% category/type match + 20% distance proximity. Warn at a configurable threshold such as 0.80.

The warning is advisory; users may continue if they genuinely have a separate issue.

10. Matching Engine Technical Design

10.1 Pipeline

Request published → determine eligible helpers → calculate distance → calculate skill fit → check availability → calculate experience/trust → weighted score → rank → store Match records → notify top candidates

10.2 Score formula

Score = 0.35 SkillFit + 0.25 DistanceScore + 0.15 AvailabilityScore + 0.15 ExperienceScore + 0.10 TrustScore

Each factor is normalized to 0–100. Final score is rounded to an integer for display.

10.3 Distance score

Example: within max radius, DistanceScore = $\max(0, 100 \times (1 - \text{distance} / \text{radius}))$. A helper at 0 km scores 100; a helper at the radius boundary scores 0.

10.4 Explainability object

```
{
  "score": 92,
  "factors": {
    "skillFit": 96,
    "distance": 94,
    "availability": 100,
    "experience": 82,
    "trust": 88,
    "reason": "Strong hardware skills and available within 0.8 km."
  }
}
```

11. Realtime Architecture

Socket.IO is used for low-latency delivery, but PostgreSQL remains authoritative.

11.1 Connection model

Authenticated client connects → server validates identity → client joins user room + relevant task rooms → server emits domain events after persistence.

11.2 Events

Event	Direction	Purpose
task:updated	Server → participants	Task status changed.
message:new	Server → task room	New chat message.
helper:availability	Server → relevant listeners	Availability changed.
match:created	Server → requester/helper	New match generated.
proof:submitted	Server → requester	Proof ready for review.
verification:updated	Server → helper	Requester verified/reworked.
live:request_created	Server → live listeners	New public/local activity.
notification:new	Server → user	New in-app notification.

11.3 Failure handling

If the socket disconnects, the client reconnects and refetches authoritative task/message state. No business state depends solely on an event being delivered.

12. Location & Maps

12.1 Data

- Store latitude and longitude for actionable requests.
- Store a human-readable location label for display.
- Do not expose exact sensitive locations unnecessarily.
- Use geocoding only when needed; cache normalized results.

12.2 Distance

MVP: Haversine calculation in backend. Scale: PostGIS geography/geometry queries and spatial indexes.

12.3 Map markers

REPORT/urgent = red; active = amber; offer = blue; resolved = green. The visual system must remain consistent across Explore Map and HelpGrid Live.

13. Proof & Media Pipeline

13.1 Upload flow

Helper selects image → client validates type/size → request signed upload or backend upload endpoint → object storage → backend stores metadata → task event emitted

13.2 Requirements

- Accept common image formats only in MVP.
- Apply a file-size limit.
- Generate storage keys that are not guessable.
- Do not trust client-provided MIME type alone.
- Store uploader, task, timestamp and object reference.
- Use optimized thumbnails for list/detail views.
- Proof cannot be deleted silently after submission; deletion should be audited.

14. Notification System

Trigger	Recipient	Message type
New high-urgency match	Helper	New nearby task available.
Helper accepted	Requester	Helper accepted your request.
Helper arrived	Requester	Helper has arrived.
Proof submitted	Requester	Resolution proof is ready.
Rework requested	Helper	Requester requested rework.
Verified	Helper	Requester confirmed resolution.
New message	Task participant	Chat notification.
No match after timeout	Requester	Widen radius / try later.

Start with in-app notifications. External providers can be added behind the same NotificationService interface.

15. Authentication & Authorization

15.1 Authentication

- Signup with email/password; phone can be added later.
- Hash passwords with Argon2id or bcrypt.
- Use secure HTTP-only cookies for session tokens where practical.
- Set expiry and rotation rules.
- Never log passwords, tokens or private message content.

15.2 Authorization matrix

Action	Requester	Helper	Org	Admin
Create request	Yes	Yes	Later	Yes
Edit own request	Yes	Yes	Later	Yes
Accept helper task	No	Yes	Later	Yes
Update task status	Participant	Participant	Later	Yes
Submit proof	No	Assigned helper	Later	Yes
Verify resolution	Requester	No	Later	Yes
View private task chat	Participants	Participants	Later	Admin policy

15.3 Security controls

- Helmet/security headers, strict CORS and input validation.
- Rate limiting for auth, messages, request creation and AI endpoints.
- Ownership checks for every mutation.
- Parameterized queries through Prisma.
- CSRF protection if cookie-based auth requires it.
- Audit logs for critical state changes.
- Secrets only through environment variables or secret manager.

16. Observability & Logging

Every API request should have a correlation/request ID. Logs should be structured JSON in production.

Log category	Examples
HTTP	method, route, status, latency, requestId
Auth	login success/failure, account events; never password/token
AI	provider latency, model version, validation failure
Matching	requestId, candidate count, scoring duration
Task	transition, actor, old/new status
Realtime	connect/disconnect, room join errors
Storage	upload success/failure, file size
System	database errors, external provider failures

Metrics

- Request creation rate
- AI processing latency/failure rate
- Match generation latency

- Acceptance rate
- Average time to first helper
- Average task completion time
- Proof submission rate
- Verification/rework rate
- Realtime connection failures

17. Performance Requirements

Area	Target / strategy
Initial page	Fast static shell; lazy-load heavy map/realtime code.
API	Aim for <500ms for normal CRUD under prototype load.
Database	Indexes on frequent filters and foreign keys.
Explore	Paginate; do not return every request.
Map	Cluster markers when count grows.
Images	Resize/compress; use thumbnails.
AI	Async where possible; show processing state.
Realtime	Join only required rooms; avoid broadcasting sensitive data.
Frontend	Code split by route and keep shared bundle small.

18. Deployment Architecture

18.1 Environment layout

Environment	Purpose
Local	Developer machine; local PostgreSQL or development database.
Staging	Integrated testing with separate database and API keys.
Production	Real deployment with managed database, object storage and provider secrets.

18.2 Deployment flow

Git push → CI checks → build → tests → deploy frontend/backend → database migration → health check

18.3 Environment variables

```

DATABASE_URL
JWT_SECRET or SESSION_SECRET
AI_API_KEY
MAP_PROVIDER_KEY
STORAGE_BUCKET / CLOUDINARY credentials
SOCKET_ORIGIN
FRONTEND_URL

```

Never commit .env files or provider secrets to source control.

18.4 Health endpoints

GET /health → API process; GET /health/db → database connectivity; optional /health/providers for external dependencies.

19. CI/CD & Repository Standards

Recommended repository structure

```
helpgrid/  
apps/web  
apps/api  
packages/shared  
prisma/  
docs/  
.github/workflows/
```

For a smaller student project, a single repository with /client and /server is acceptable. The technical boundary should still remain clear.

CI checks

- Install dependencies
- Lint
- Type check where TypeScript is used
- Unit tests
- API/integration tests
- Build frontend
- Build backend
- Database migration validation

Git conventions

Use feature branches, focused commits and pull requests. Suggested commit style: feat:, fix:, refactor:, test:, docs:, chore:.

20. Technical Testing Strategy

Level	Examples
Unit	Match score, Haversine distance, state transitions, validators.
Integration	Request publish triggers matching and notifications.
API	Auth, authorization, CRUD, proof and verification.
Realtime	Message delivery, reconnect and state resync.
AI	Schema validity, null handling, fallback behavior, prompt regression fixtures.
Security	Unauthorized access, IDOR/ownership, upload validation, rate limits.
E2E	Create → AI → publish → match → accept → task → proof → verify.
Load	Explore endpoint, match generation and websocket connections under expected demo scale.

20.1 Critical test scenario

Given request “Lab 3 PCs won't boot” and an available helper with hardware skill at 0.8 km, the system should create a valid REPORT, classify it as high urgency, produce a match, allow acceptance, persist task state, accept proof, and permit requester verification.

21. Failure & Recovery Design

Failure	Expected behavior
AI unavailable	Publish using deterministic fallback; label no AI suggestion.
Map provider unavailable	Allow manual location text or retry; do not lose draft.
Socket disconnected	Show reconnect state and refetch authoritative data.
Upload failed	Keep task open; allow retry without duplicate proof record.

Failure	Expected behavior
Database transient error	Return safe error; do not partially commit critical transition.
Notification provider down	Persist in-app notification; retry external delivery asynchronously.
Match service error	Request remains published; user can retry matching.

22. Basic Threat Model

Threat	Example	Control
IDOR	User edits another user's request	Ownership + authorization checks.
Credential theft	Stolen session/token	Secure cookies, expiry, rotation, HTTPS.
XSS	Malicious request/message content	Sanitize/escape rendered text.
Upload abuse	Executable or huge file	MIME/extension/size validation + object storage rules.
Spam	Automated request/message creation	Rate limiting + later reputation/moderation.
Location leakage	Exact home location visible	Precision controls + restricted visibility.
AI prompt injection	User text attempts to alter system instructions	Strict system prompt + structured output validation + no tool privileges.
Data exposure	Logs contain secrets	Redaction policy and structured logging.

23. Implementation Roadmap

Sprint	Technical deliverables
1	Repo, React app, Express API, Prisma, PostgreSQL, env setup, CI.
2	Auth, user profile, role onboarding, shared types.
3	Request model/API, create flow, validation and Explore.
4	AI understanding endpoint, schema validation, review UI.
5	Map integration, location storage and duplicate detection.
6	Helper availability, matching engine and match explanation.
7	Task state machine, notifications and Socket.IO.
8	Messaging, proof uploads and verification.
9	HelpGrid Live, system states, performance and security hardening.
10	E2E tests, deployment, demo seed data and documentation.

Recommended engineering order

Build the domain and state machine before polishing visual details. Once the request lifecycle is stable, connect UI screens to the same API/data model rather than creating page-specific mock objects.

24. Technical Acceptance Criteria

- A new user can register and authenticate.
- A requester can create all four request types.
- A request can be created from natural language without a long form.
- AI output is schema-validated and editable.
- Requests can be located and filtered by distance/type/urgency.
- Available helpers can receive explainable matches.
- Only eligible helpers can accept a task.
- Task state transitions are enforced server-side.
- Messages are persisted and delivered realtime when connected.
- Proof files are securely stored with metadata.
- Requester verification changes the final state.
- Socket failure does not corrupt task state.
- Critical data is persisted in PostgreSQL.
- Unauthorized users cannot mutate other users' resources.
- The canonical Lab 3 request remains consistent throughout the full demo.
- The system can run with AI disabled using deterministic fallbacks.

25. Technical Definition of Done

The technical implementation is complete when the full HelpGrid loop works against a real backend and database, critical state changes are transactional and authorized, realtime features recover gracefully, AI output is validated, proof is persisted, verification is enforced, and the application can be deployed from a reproducible repository.

Final architecture summary

React UI → Express API → Domain Services → Prisma/PostgreSQL
 ↳ AI Service
 ↳ Matching Engine

↳ Socket.IO
↳ Maps / Storage / Notifications

The most important technical rule is simple: the Request and Task state in PostgreSQL are the source of truth. AI recommends, Socket.IO communicates, and the frontend renders — but none of them independently owns the final business state.